RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM 602 105

CS23621

MOBILE APPLICATION DEVELOPMENT LABORATORY

LAB MANUAL

Name :	SAHANA R
Year / Branch / Section :	III/INFORMATION TECHNOLOGY
University Register No. :	2116231001173
College Roll No. :	231001173
Semester :	VI SEMESTER
Academic Year :	2025-2026 .

RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……SAHANA R……

Academic Year: ……2025-2026…… Semester: …VI… Branch: ……IT…

Register No.	**2116231001173**

Certified that this is the bonafide record of work done by the above student in the* CS23621 – MOBILE APPLICATION DEVELOPMENT *Laboratory during the academic year 2025- 2026

Signature of Faculty in-charge

Submitted for the Practical Examination held on……

Internal Examiner **External Examiner**

INDEX

EX.NO	DATE	NAME OF THE EXPERIMENT	SIGN	GITHUB QR CODE
1		Design a simple Android app using Kotlin basics. Display a welcome message on the screen. Use variables, data types, and control statements.		
2		Create an Android app to perform arithmetic operations. Use functions for addition, subtraction, multiplication, and division. Display results based on user input.		
3		Develop an app to manage student details. Use classes and objects in Kotlin. Display student information on the screen.		
4		Build your first Android app. Add a button and handle click events. Show a toast message when clicked.		
5		Design a login screen using layouts. Use LinearLayout or ConstraintLayout. Include username and password fields.		
6		Create navigation between two screens. Move from Login screen to Home screen. Use intents or navigation components.		
7		Demonstrate Activity and Fragment lifecycle. Log each lifecycle method. Display lifecycle changes on screen.		
8		Develop an app using MVVM architecture. Create UI layer with ViewModel. Display a list of items.		
9		Build an app with data persistence. Use Room database to store data. Retrieve and display stored data.		
10		Create an app using RecyclerView. Display a list of items with images. Handle item click events.		
11		Develop an app to connect to the internet. Fetch data from an API. Display the fetched data.		
12		Implement Repository Pattern. Use WorkManager for background tasks. Fetch and cache data efficiently.		

13		Design a user-friendly app UI. Apply colors, spacing, and alignment. Ensure good usability and user experience.		

Ex.No. 01

WELCOME APP

AIM:

To design a simple Android app using Kotlin basics to display a welcome message using variables, data types, and control statements.

CODE:

MainActivity.kt:

```
package com.example.mad import
android.os.Bundle import android.widget.TextView
import
androidx.appcompat.app.AppCompatActivity class
MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState)
// Make sure this matches your XML file name
setContentView(R.layout.activity_main) val textView =
findViewById<TextView>(R.id.textView) textView.text =
"Welcome Shakthi!"}}
```

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
 <androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content
        "
        android:layout_height="wrap_conten
        t" android:text="Hello"
        android:textSize="26sp"
app:layout_constraintTop_toTopOf="parent"                app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent"/>


      </androidx.constraintlayout.widget.ConstraintLayout>
```

OUTPUT:

No:

RESULT:

The app successfully displays a dynamic welcome message on the screen based on user input or predefined values.

Ex 02

ARITHMETIC OPERATIONS APP

AIM:

To develop an Android app to perform basic arithmetic operations using Kotlin functions.

CODE:

MainActivity.kt:

```
package com.example.mad import
android.os.Bundle import android.widget.* import
androidx.appcompat.app.AppCompatActivity class
MainActivity : AppCompatActivity() {
fun add(a: Double, b: Double) = a + b
fun sub(a: Double, b: Double) = a - b
fun mul(a: Double, b: Double) = a * b
fun div(a: Double, b: Double) = if (b != 0.0) a / b else "Cannot divide by 0"
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main) val num1 =
findViewById<EditText>(R.id.num1) val num2 =
findViewById<EditText>(R.id.num2) val result =
findViewById<TextView>(R.id.result) val addBtn =
findViewById<Button>(R.id.addBtn) val subBtn =
findViewById<Button>(R.id.subBtn) val mulBtn =
findViewById<Button>(R.id.mulBtn) val divBtn =
findViewById<Button>(R.id.divBtn) addBtn.setOnClickListener { val a =
num1.text.toString().toDoubleOrNull() val b =
num2.text.toString().toDoubleOrNull()
if (a != null && b != null) { result.text
= "Result: ${add(a, b)}"
} else {
result.text = "Enter valid numbers"
}
}
subBtn.setOnClickListener { val a =
num1.text.toString().toDoubleOrNull() val b =
num2.text.toString().toDoubleOrNull()
if (a != null && b != null) { result.text
= "Result: ${sub(a, b)}"
} else {
result.text = "Enter valid numbers"
}
}
mulBtn.setOnClickListener { val a =
num1.text.toString().toDoubleOrNull() val b =
num2.text.toString().toDoubleOrNull()
```

No:

```
if (a != null && b != null) { result.text
= "Result: ${mul(a, b)}"
} else {
result.text = "Enter valid numbers"
}
}
divBtn.setOnClickListener  {
val a = num1.text.toString().toDoubleOrNull() val
b = num2.text.toString().toDoubleOrNull()
if (a != null && b != null) { result.text
= "Result: ${div(a, b)}"
} else {
result.text = "Enter valid numbers"
}
}
}
}
```

OUTPUT:

RESULT:

The app successfully performs addition, subtraction, multiplication, and division and displays correct results based on user input.

03

STUDENT DETAILS

To create an Android application using classes and objects to manage and display student details.

CODE:

MainActivity.kt

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto"
 android:layout_width="match_parent"
 android:layout_height="match_parent">
<TextView android:id="@+id/textView"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Student Details"
android:textSize="20sp"
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintBottom_toBottomOf="pare
nt"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"/>
</androidx.constraintlayout.widget.ConstraintLayout>
```

OUTPUT:

Ex. No:

AIM:

RESULT:

The app successfully stores student data using a class and displays the details on the screen.

04

BUTTON CLICK

To build an Android app with a button and handle click events to display a Toast message.

CODE:

MainActivity.kt package com.example.mad import

android.os.Bundle import android.widget.Button

import android.widget.Toast import

androidx.appcompat.app.AppCompatActivity class

MainActivity : AppCompatActivity() { override fun

onCreate(savedInstanceState: Bundle?) {

super.onCreate(savedInstanceState)

setContentView(R.layout.activity_main) val button =

findViewById<Button>(R.id.button)

button.setOnClickListener {

Toast.makeText(this, "Button Clicked!", Toast.LENGTH_SHORT).show()

}

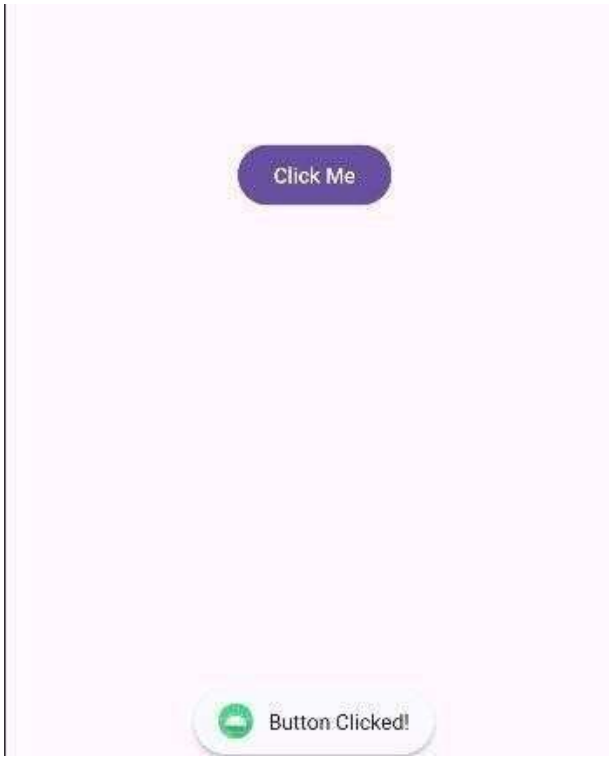
}

}

OUTPUT:

Ex. No:

AIM:



RESULT:

The app successfully shows a Toast message when the button is clicked.

05

LOGIN SCREEN UI

To design a login screen using layouts like LinearLayout or ConstraintLayout with username and password fields.

CODE:

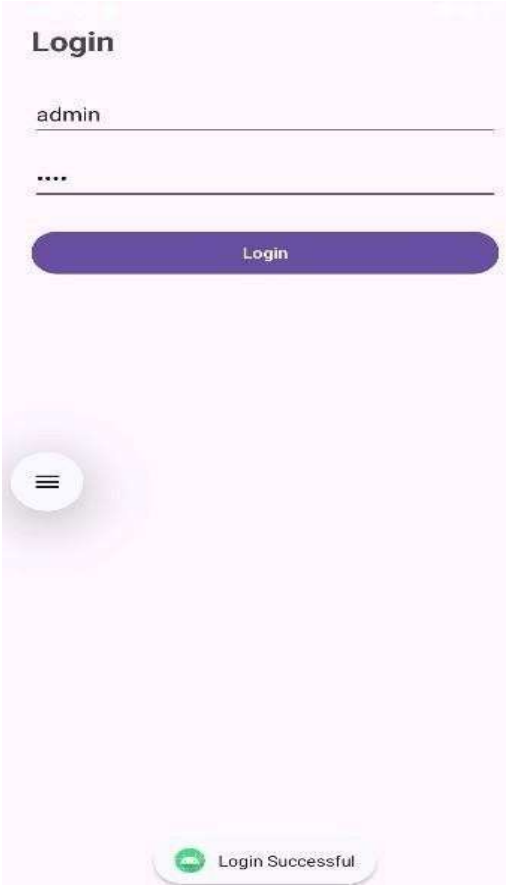
MainActivity.kt:

```
package com.example.mad import android.os.Bundle

import android.widget.* import

androidx.appcompat.app.AppCompatActivity class

MainActivity : AppCompatActivity() { override fun

onCreate(savedInstanceState: Bundle?) {

super.onCreate(savedInstanceState)

setContentView(R.layout.activity_main) val username =

findViewById<EditText>(R.id.username) val password

= findViewById<EditText>(R.id.password) val loginBtn

= findViewById<Button>(R.id.loginBtn)

loginBtn.setOnClickListener { val user =

username.text.toString() val pass =

password.text.toString() if (user == "admin" && pass ==

"1234") {

Toast.makeText(this, "Login Successful", Toast.LENGTH_SHORT).show()

} else {

Toast.makeText(this, "Invalid Credentials",

Toast.LENGTH_SHORT).show()

}

}
}

}
```

Ex. No:

AIM:

OUTPUT:



RESULT:

A user-friendly login interface is created with proper alignment, spacing, and input fields.

06

NAVIGATION

To implement navigation between two screens using intents.

CODE:

MainActivity.kt:

```
package com.example.mad import android.content.Intent import android.os.Bundle import android.widget.Button import androidx.appcompat.app.AppCompatActivity class MainActivity : AppCompatActivity() { override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState) setContentView(R.layout.activity_main) val loginBtn = findViewById<Button>(R.id.loginBtn) loginBtn.setOnClickListener { val intent = Intent(this, HomeActivity::class.java) startActivity(intent)}}}
```

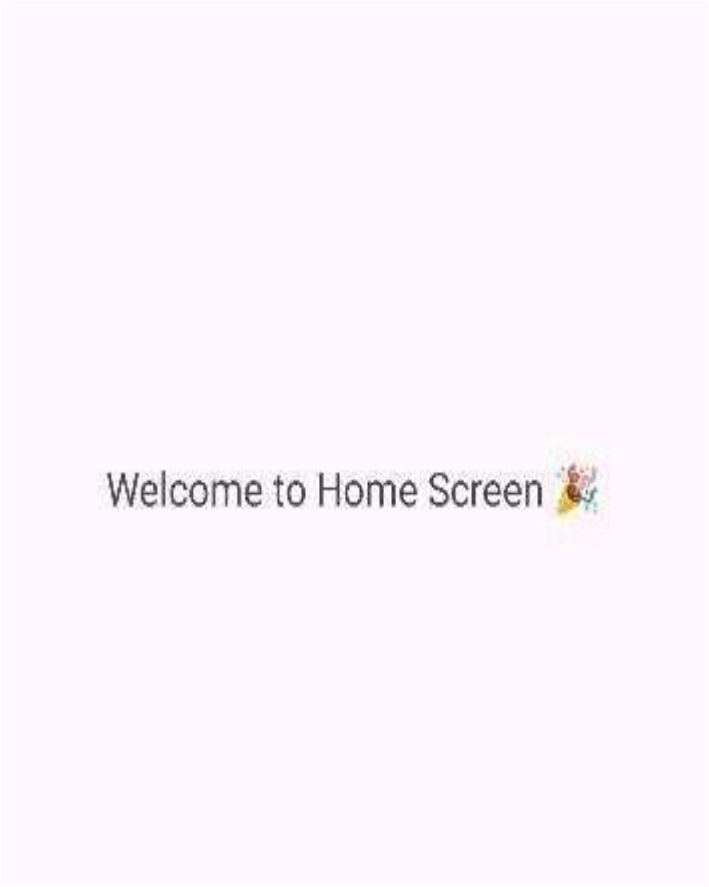
HomeActivity.kt:

```
package com.example.mad import android.os.Bundle import androidx.appcompat.app.AppCompatActivity class HomeActivity : AppCompatActivity() { override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState) setContentView(R.layout.activity_home)}}
```

OUTPUT:

Ex. No:

AIM:



RESULT:

The app successfully navigates from the login screen to the home screen on button click.

07

ACTIVITY LIFECYCLE

To demonstrate the lifecycle methods of an Activity and observe state changes.

CODE:

MainActivity.kt package com.example.mad import

android.os.Bundle import android.util.Log import

android.widget.Button import

android.widget.TextView import

androidx.appcompat.app.AppCompatActivity class

MainActivity : AppCompatActivity() { lateinit var

textView: TextView var isPaused = false fun

updateState(state: String) {

textView.append("\n$state")

Log.d("Lifecycle", state)

} override fun onCreate(savedInstanceState: Bundle?) {

super.onCreate(savedInstanceState)

setContentView(R.layout.activity_main) textView =

findViewById(R.id.textView) val triggerBtn =

findViewById<Button>(R.id.triggerBtn) val closeBtn =

findViewById<Button>(R.id.closeBtn)

updateState("onCreate")

// Trigger lifecycle simulation
triggerBtn.setOnClickListener { if

(!isPaused) {

updateState("Simulating

Ex. No:

AIM:

onPause")

updateState("Simulating onStop")

isPaused = true triggerBtn.text =

"Resume App"

} else { updateState("Simulating

onRestart") updateState("Simulating

onStart") updateState("Simulating

onResume") isPaused = false

triggerBtn.text = "Pause App"}} //

Real close (actual lifecycle)

closeBtn.setOnClickListener {

updateState("Closing App →

finish()") finish()}} override fun

onStart() { super.onStart()

updateState("onStart")} override fun

onResume() { super.onResume()

updateState("onResume")} override

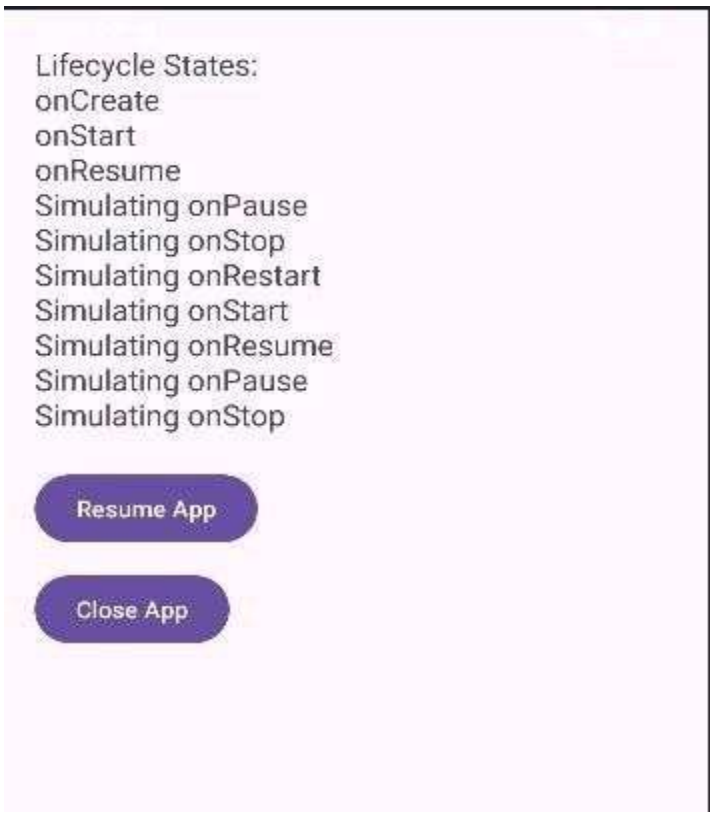
fun onPause() { super.onPause()

updateState("onPause")} override fun

onStop() { super.onStop()

```
updateState("onStop")}
override fun onDestroy() {
super.onDestroy()
Log.d("Lifecycle", "onDestroy")}}
```

OUTPUT:



RESULT:

Lifecycle methods like onCreate(), onStart(), onResume(), onPause(), onStop(), and onDestroy() are successfully logged and observed.

Ex. No: 08

MVVM ARCHITECTURE

AIM:

To develop an Android app using MVVM architecture with ViewModel and LiveData.

CODE:

MainActivity.kt:

```
package com.example.mad import android.os.Bundle
import android.widget.TextView import
androidx.appcompat.app.AppCompatActivity import
androidx.lifecycle.ViewModelProvider class
MainActivity : AppCompatActivity() { lateinit var
viewModel: MyViewModel lateinit var textView:
TextView override fun onCreate(savedInstanceState:
Bundle?) { super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main) textView =
findViewById(R.id.textView)
// Get ViewModel viewModel =
ViewModelProvider(this)[MyViewModel::class.java]
// Observe LiveData
viewModel.data.observe(this) { list ->
textView.text = list.joinToString("\n")}}}
```

MyViewModel.kt

```
package
com.example.mad import
androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
```

```
class MyViewModel : ViewModel() { val
data = MutableLiveData<List<String>>()
init {
// Sample data (Model) data.value
= listOf(
"Apple",
"Banana",
"Orange",
"Mango"
)}}
```

OUTPUT:

RESULT:

The app successfully separates UI and data logic and displays a list of items using ViewModel..

Ex. No: 09

ROOM DATABASE

AIM:

To implement local data persistence using Room Database.

CODE:

MainActivity.kt package com.example.mad import android.os.Bundle import android.widget.* import androidx.appcompat.app.AppCompatActivity import androidx.room.Room class MainActivity : AppCompatActivity() { lateinit var db: AppDatabase lateinit var textView: TextView override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState) setContentView(R.layout.activity_main) val editText = findViewById<EditText>(R.id.editText) val saveBtn = findViewById<Button>(R.id.saveBtn) textView = findViewById(R.id.textView) db = Room.databaseBuilder(applicationContext, AppDatabase::class.java, "user-db").allowMainThreadQueries().build () saveBtn.setOnClickListener { val name = editText.text.toString() if (name.isNotEmpty()) { db.userDao().insert(User(name = name)) displayData() editText.text.clear()}}

```
displayData()}} fun displayData()
{ val users = db.userDao().getAll()
textView.text =
users.joinToString("\n") { it.name
}}}
```

User.kt

```
package
com.example.mad
import
androidx.room.Entity
import
androidx.room.PrimaryKey
@Entity data
class User(
@PrimaryKey(autoGenerate = true) val id: Int = 0, val
name: String
)
```

UserDAO.kt

```
package
com.example.mad
import
androidx.room.Dao
import
androidx.room.Insert
import
androidx.room.Query
@Dao
interface UserDao {
@Insert fun
insert(user: User)
@Query("SELECT * FROM User")
fun getAll(): List<User>
}
```

Appdatabase.kt

```
package
com.example.mad
import
```

```
androidx.room.Database import
androidx.room.RoomDatabase
@Database(entities = [User::class], version =
1) abstract class AppDatabase :
RoomDatabase() { abstract fun userDao():
UserDao
}
```

OUTPUT:

RESULT:

The app successfully stores, retrieves, and displays data using Room (Entity, DAO, Database).

Ex. No: 10

RECYCLE VIEW

AIM:

To create an app that displays a list of items using RecyclerView and handles item click events..

CODE:

MainActivity.kt package com.example.mad import android.os.Bundle import androidx.appcompat.app.AppCompatActivity import androidx.recyclerview.widget.LinearLayoutManager import androidx.recyclerview.widget.RecyclerView class MainActivity : AppCompatActivity() { override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState) setContentView(R.layout.activity_main) val recyclerView = findViewById<RecyclerView>(R.id.recyclerView)

```
val list = listOf(
Item("Apple", R.drawable.apple),
Item("Banana", R.drawable.apple),
Item("Orange", R.drawable.apple),
Item("Mango", R.drawable.apple),
Item("Grapes", R.drawable.apple)
)
recyclerView.layoutManager = LinearLayoutManager(this) recyclerView.adapter
= MyAdapter(list)
}
}
```

MyAdapter.kt package com.example.mad import android.view.LayoutInflater import android.view.View import android.view.ViewGroup import android.widget.* import

```
androidx.recyclerview.widget.RecyclerView class
MyAdapter(private val list: List<Item>) :
RecyclerView.Adapter<MyAdapter.ViewHolder>() { class
ViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) { val
textView: TextView = itemView.findViewById(R.id.itemText) val
imageView: ImageView = itemView.findViewById(R.id.itemImage)
} override fun onCreateViewHolder(parent: ViewGroup, viewType:
Int):
ViewHolder { val view =
LayoutInflater.from(parent.context)
.inflate(R.layout.item_layout, parent, false)
return ViewHolder(view)
} override fun onBindViewHolder(holder: ViewHolder, position: Int)
{ val item = list[position] holder.textView.text = item.name
holder.imageView.setImageResource(item.image)
holder.itemView.setOnClickListener { Toast.makeText(
holder.itemView.context,
"Clicked: ${item.name}",
Toast.LENGTH_SHORT
).show()
} } override fun getItemCount(): Int =
list.size
}
```

OUTPUT:

RESULT:

The app successfully displays a scrollable list with images/text and responds to item clicks.

11

API FETCH

To develop an app that connects to the internet and fetches data from an API.

CODE:

MainActivty.kt:

```
package com.example.mad import android.os.Bundle import
android.widget.TextView import
androidx.appcompat.app.AppCompatActivity import java.net.URL
import kotlin.concurrent.thread class MainActivity :
AppCompatActivity() { override fun onCreate(savedInstanceState:
Bundle?) { super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main) val textView =
findViewById<TextView>(R.id.textView) thread { try { val url =
URL("https://jsonplaceholder.typicode.com/posts") val result =
url.readText() runOnUiThread { textView.text = result.take(500) //
show partial data
}
} catch (e: Exception) { runOnUiThread {
textView.text = "Error: ${e.message}"}}}
```

OUTPUT:

Ex. No:

AIM:


```
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati
excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae
consequuntur expedita et cum\nreprehenderit
molestiae ut ut quas totam\nnostrum rerum est
autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil
reprehenderit dolor beatae ea dolores neque\nfugiat
blanditiis voluptate p
```

RESULT:

The app successfully retrieves data from an API and displays it on the screen.

12

WORK MANAGER

To implement background task execution using WorkManager.

CODE:

MainActivity.kt:

```
package com.example.mad import android.os.Bundle import
android.widget.Button import android.widget.TextView import
androidx.appcompat.app.AppCompatActivity import androidx.work.* class
MainActivity : AppCompatActivity() { override fun
onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main) val btn =
findViewById<Button>(R.id.button) val statusText =
findViewById<TextView>(R.id.statusText) btn.setOnClickListener { val
workRequest =
OneTimeWorkRequest.Builder(MyWorker::class.java).build()
WorkManager.getInstance(this).enqueue(workRequest)
// OBSERVE STATUS
WorkManager.getInstance(this)
.getWorkInfoByIdLiveData(workRequest.id)
.observe(this) { workInfo -> if (workInfo != null)
{ statusText.text = "Status:
${workInfo.state}"}}}}}
```

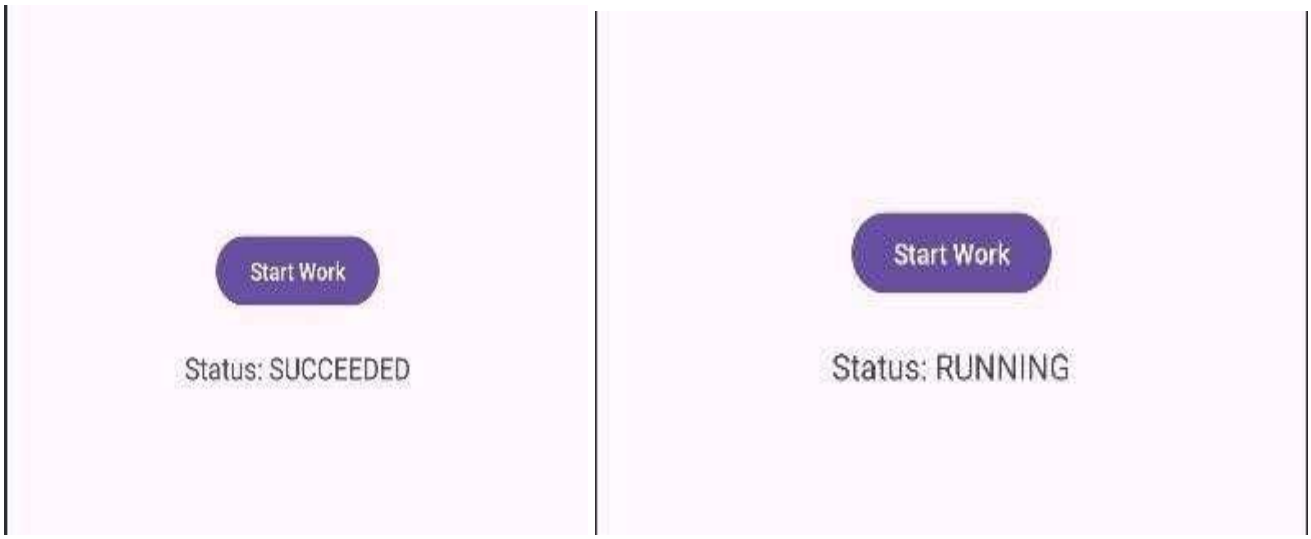
MyWorker.kt:

Ex. No:

AIM:

```
package com.example.mad import
android.content.Context import android.util.Log
import android.widget.Toast import
androidx.work.Worker import
androidx.work.WorkerParameters class
MyWorker(context: Context, params:
WorkerParameters)
: Worker(context,
params) { override fun
doWork(): Result {
Log.d("WorkManager", "Task Running...")
Thread.sleep(2000) // simulate
work Log.d("WorkManager",
"Task Completed") return
Result.success()}}
```

OUTPUT:



RESULT:

The app successfully executes background tasks and updates the UI/log with task status.

13

UI DESIGN

To design a user-friendly Android interface with proper colors, spacing, and alignment.

CODE:

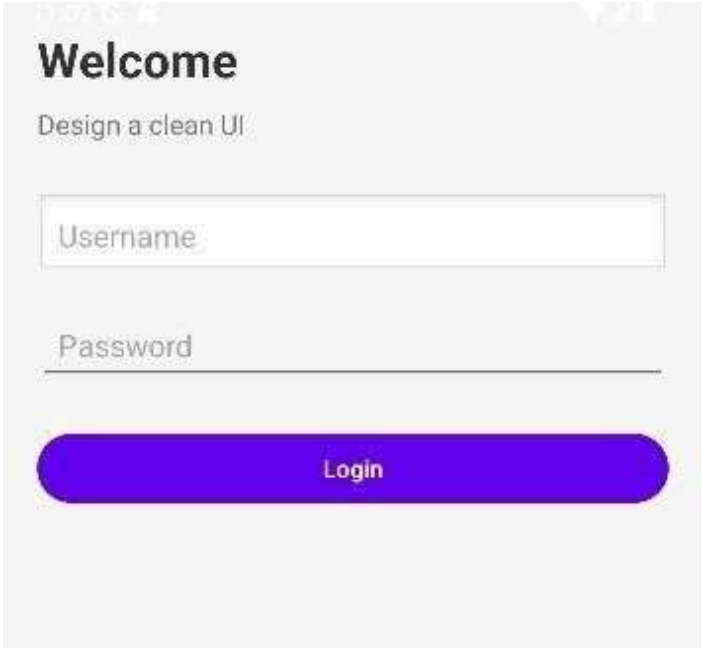
MainActivity.kt:

```
package com.example.mad import
android.os.Bundle import android.widget.* import
androidx.appcompat.app.AppCompatActivity class
MainActivity : AppCompatActivity() { override fun
onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main) val btn =
findViewById<Button>(R.id.loginBtn)
btn.setOnClickListener {
 Toast.makeText(this, "UI Designed Successfully!",
Toast.LENGTH_SHORT).show()
 }
 }
}
```

OUTPUT:

Ex. No:

AIM:



RESULT:

A visually appealing and user-friendly UI is created following design principles and usability guidelines.

PROJECT REPORT

APTIPREP QUIZ APP

1. Introduction:

Mobile applications have become an essential part of modern learning. Students increasingly rely on smartphones for practicing aptitude and improving problem-solving skills. However, many existing platforms are either complex, paid, or not personalized.

The **AptiPrep Quiz App** is an Android application developed using **Kotlin in Android Studio**. It is designed to help users practice aptitude questions in a simple, interactive, and user-friendly way. The app allows users to select topics, attempt quizzes, track scores, and view their performance history.

This application focuses on providing **easy accessibility, topic-wise practice, and progress tracking**, making it suitable for beginners preparing for competitive exams or placements.

2. Problem Statement:

Many students face difficulties in preparing for aptitude tests due to:

- Lack of simple and free practice platforms
- No proper tracking of progress over time
- Limited topic-wise practice
- Overly complex or distracting applications
- No personalization for individual users

There is a need for a **lightweight, easy-to-use mobile application** that enables users to practice aptitude questions and track their improvement efficiently.

3. Proposed System:

The proposed system is an Android-based quiz application that:

- Allows users to **enter their name** for personalization
- Provides **topic-wise quizzes** (Quantitative, Logical, Verbal)
- Displays questions with multiple-choice answers
- Enables navigation using **Next and Previous buttons**
- Calculates and displays the score at the end
- Stores user performance using **SharedPreferences**

- Maintains a **history of attempts**
- Displays top score and performance tracking
- The system is designed to be **simple, fast, and user-friendly**.

4. Methodology:

The development of the application follows a structured approach:

i. Requirement Analysis

- Identified need for a simple aptitude practice app
- Defined features like quiz, scoring, and history **ii. Design**
- UI designed using XML layouts
- Screens include:
- Login Screen
- Topic Selection
- Quiz Screen
- Result Screen
- History Screen **iii. Implementation**
- Developed using **Kotlin** in **Android Studio**
- Questions stored in a **JSON file** inside assets
- Data loaded using a QuestionBank class
- Score and history stored using **SharedPreferences iv. Testing**
- Tested navigation between screens
- Verified score calculation
- Checked history storage and retrieval
- Fixed UI and runtime errors

v. Deployment

- App runs on Android devices (API 24 and above)

5. Unique Features:

The application includes several unique and useful features:

- **User Personalization**
  Users enter their name, and their scores are tracked individually.
- **Topic-wise Quiz Selection** Questions are categorized into:

  i. Quantitative Aptitude
  ii. Logical Reasoning iii. Verbal Ability

- **Previous & Next Navigation**
  Users can move back to change answers before submitting.
- **Progress Tracking**
  Each attempt is stored and displayed in a history page.
- **Top Score Storage**
  Highest score is saved and updated automatically.
- **History View (Numbered List)**
  Users can see all past attempts with scores.
- **Clean & Elegant UI**
  Light theme, simple navigation, and minimal distractions.

6. Source Code:

Activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
  <androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <Button android:id="@+id/startBtn"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" android:text="Start
      Quiz"
      app:layout_constraintTop_toTopOf="parent"
      app:layout_constraintBottom_toBottomOf="parent"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintEnd_toEndOf="parent"/>

  </androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt:

```
package com.example.aptiprep

  import android.content.Intent import android.os.Bundle import
  android.widget.Button
```

```
import androidx.appcompat.app.AppCompatActivity class MainActivity :

AppCompatActivity() {

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState) setContentView(R.layout.activity_main) val

    startBtn = findViewById<Button>(R.id.startBtn)

                    startBtn.setOnClickListener {
      startActivity(Intent(this, TopicActivity::class.java)) }
  }
}
```

LoginActivity.kt:

```
package com.example.aptiprep

import android.content.Intent import android.os.Bundle import
android.widget.Button import android.widget.EditText
import androidx.appcompat.app.AppCompatActivity class LoginActivity :

AppCompatActivity() {

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState) setContentView(R.layout.activity_login)

    val nameInput = findViewById<EditText>(R.id.nameInput) val startBtn =
    findViewById<Button>(R.id.startBtn)

                    startBtn.setOnClickListener {
                      val name = nameInput.text.toString()
      if (name.isEmpty()) {
                       nameInput.error = "Enter your name"
                       return@setOnClickListener
      }

      val prefs = getSharedPreferences("quiz", MODE_PRIVATE)
      prefs.edit().putString("username", name).apply()

      startActivity(Intent(this, TopicActivity::class.java)) finish()
    }
  }
}
```

TopicActivity.kt:

```
package com.example.aptiprep
```

```
import android.content.Intent import android.os.Bundle import
android.widget.Button import android.widget.ImageView import
android.widget.TextView import androidx.appcompat.app.AppCompatActivity class
TopicActivity : AppCompatActivity() {

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState) setContentView(R.layout.activity_topic)

    val quantBtn = findViewById<Button>(R.id.quantBtn) val logicalBtn =
    findViewById<Button>(R.id.logicalBtn) val verbalBtn =
    findViewById<Button>(R.id.verbalBtn) val historyBtn =
    findViewById<TextView>(R.id.historyBtn) val backBtn =
    findViewById<ImageView>(R.id.backBtn)

    // cQ )➔ Back to Login backBtn.setOnClickListener {
      startActivity(Intent(this, LoginActivity::class.java)) finish()
    }

    // Ilh _ History historyBtn.setOnClickListener {
      startActivity(Intent(this, HistoryActivity::class.java)) }
    // Topics
    quantBtn.setOnClickListener { openQuiz("quant") }
  logicalBtn.setOnClickListener { openQuiz("logical") }
  verbalBtn.setOnClickListener { openQuiz("verbal") } }

  private fun openQuiz(topic: String) { val intent = Intent(this,
    QuizActivity::class.java) intent.putExtra("topic", topic)
    startActivity(intent)
  }
}
```

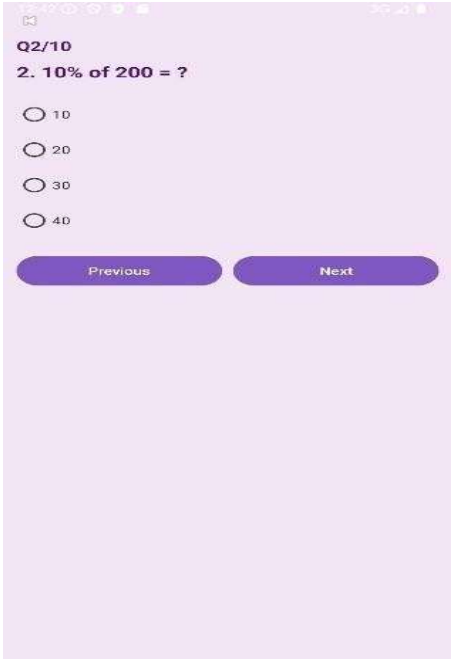
7. Output Images:

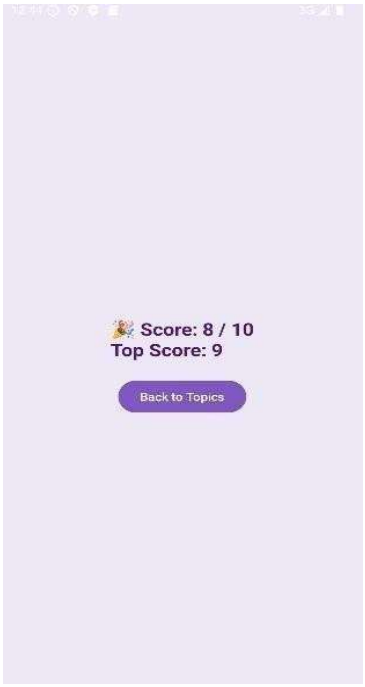
Login Screen:

Topic Selection Screen:

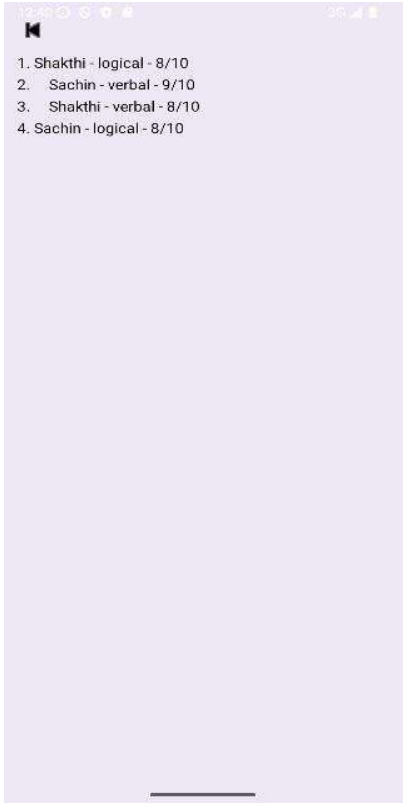
Quiz Screen:

Result Screen:





History Screen:



8. Conclusion:

The AptiPrep Quiz App successfully provides a simple and effective platform for practicing aptitude questions. It allows users to:

- Improve their problem-solving skills
- Track their progress over time
- Practice topic-wise questions

The application is lightweight, easy to use, and suitable for beginners. It demonstrates the use of Android development concepts such as activities, layouts, data storage, and user interaction.

This project can be further enhanced by adding features like online databases, timers, and leaderboards.

9. References:

- **https://www.researchgate.net/publication/379102977**
- **https://www.researchgate.net/publication/368755973**
- **https://www.ijiet.org**